

Experiment No.	2
Experiment Title	Email Spam/Ham Classification using Naïve Bayes and KNN ₁
Student Name	Pranav N
Register Number	3122247001044
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	Aug 2026

1 Objective

- Build spam classifiers using Naïve Bayes and K-Nearest Neighbors (KNN).
- Compare Gaussian, Multinomial, and Bernoulli Naïve Bayes variants.
- Study the effect of varying the number of neighbors k in KNN.
- Compare the KDTree and BallTree nearest-neighbor search algorithms.
- Optimize KNN hyperparameters using GridSearchCV and RandomizedSearchCV.
- Evaluate the computational (training/prediction) time of each algorithm.
- Evaluate model performance using 5-fold Cross Validation.

2 Problem Statement

Given a dataset of emails represented by 57 numeric features describing word/character frequencies and capital-letter run-length statistics, the task is to classify each email as **spam** (1) or **ham/not-spam** (0).

- **Input:** 57 continuous numeric features per email (word frequencies, character frequencies, and capital-run-length statistics).
- **Process:** Load and clean the data, perform EDA, scale features appropriately for each classifier, train and compare Naïve Bayes variants (Gaussian, Multinomial, Bernoulli) and KNN, tune KNN hyperparameters via GridSearchCV and RandomizedSearchCV, compare KDTree vs. BallTree, and evaluate all models using 5-fold cross validation.
- **Output:** A binary class label (spam / ham) for each email, along with a comparative analysis of classifier accuracy, robustness, and computational cost.

Theory

Naïve Bayes is a probabilistic classifier based on Bayes' theorem, which assumes conditional independence between features given the class:

$$P(C | X) = \frac{P(X | C) P(C)}{P(X)} \quad (1)$$

where C is the class label (spam/ham) and X is the feature vector of an email. Three variants were compared: **Gaussian NB** (assumes features follow a normal distribution within each class), **Multinomial NB** (models discrete counts / frequencies, suited to word-frequency style features), and **Bernoulli NB** (models binary presence/absence of features).

KNN is a lazy, instance-based learning algorithm that classifies a query sample based on the majority class among its k nearest neighbors in feature space, using a distance metric such as Euclidean or Manhattan distance. Since KNN performs no explicit training phase, all computation is deferred to prediction time, when distances to (a subset of) the training points must be computed. KDTree and BallTree are spatial indexing structures that accelerate this nearest-neighbor search compared to a brute-force scan.

3 Dataset Description

Dataset Name	Spambase Dataset
Dataset Source	UCI Machine Learning Repository / Kaggle (https://www.kaggle.com/datasets/somesh24/spambase)
Number of Samples	4601
Number of Features	57 numeric features (word frequency, character frequency, capital-run-length statistics)
Number of Classes	2 (Spam = 1: 1813 samples; Ham = 0: 2788 samples)
Missing Values	0 (none found)
Train-Test Split	75% train (3450 samples) / 25% test (1151 samples), stratified, <code>random_state=42</code>

Note: the notebook was executed using the header-less UCI distribution of the dataset (`spambase.data`, 57 features + 1 label column), so features are referenced by their positional names `f0–f56` throughout this report, exactly as produced by the executed pipeline.

4 Exploratory Data Analysis

EDA was performed on the raw 57-feature email dataset prior to any scaling.

- **Dataset overview:** 4601 emails $\times$ 57 numeric features + 1 binary target (**spam**).
- **Missing value analysis:** No missing values were found in any column; no imputation was required.
- **Class distribution:** 2788 ham emails (60.6%) and 1813 spam emails (39.4%) – a moderate class imbalance.
- **Correlation matrix:** Computed for the top 15 features most correlated with the target, most inter-feature correlations are weak-to-moderate, with feature `f24` showing a notable negative correlation with several other top features.

- **Feature distribution:** The top 4 most target-correlated features (f20, f22, f6, f52) are strongly right-skewed, with most values concentrated near zero and a long tail of larger values, more pronounced for spam emails.



Figure 1: Class distribution of the Spambase dataset (Ham = 0, Spam = 1).

Inference: The dataset is moderately imbalanced, with roughly 60% ham and 40% spam emails. This imbalance is not severe enough to require resampling techniques, but it does justify reporting Precision, Recall, F1-score, and ROC-AUC alongside Accuracy, since accuracy alone can be misleading on imbalanced data. Stratified train-test splitting was used to preserve this class ratio in both the training and test sets.

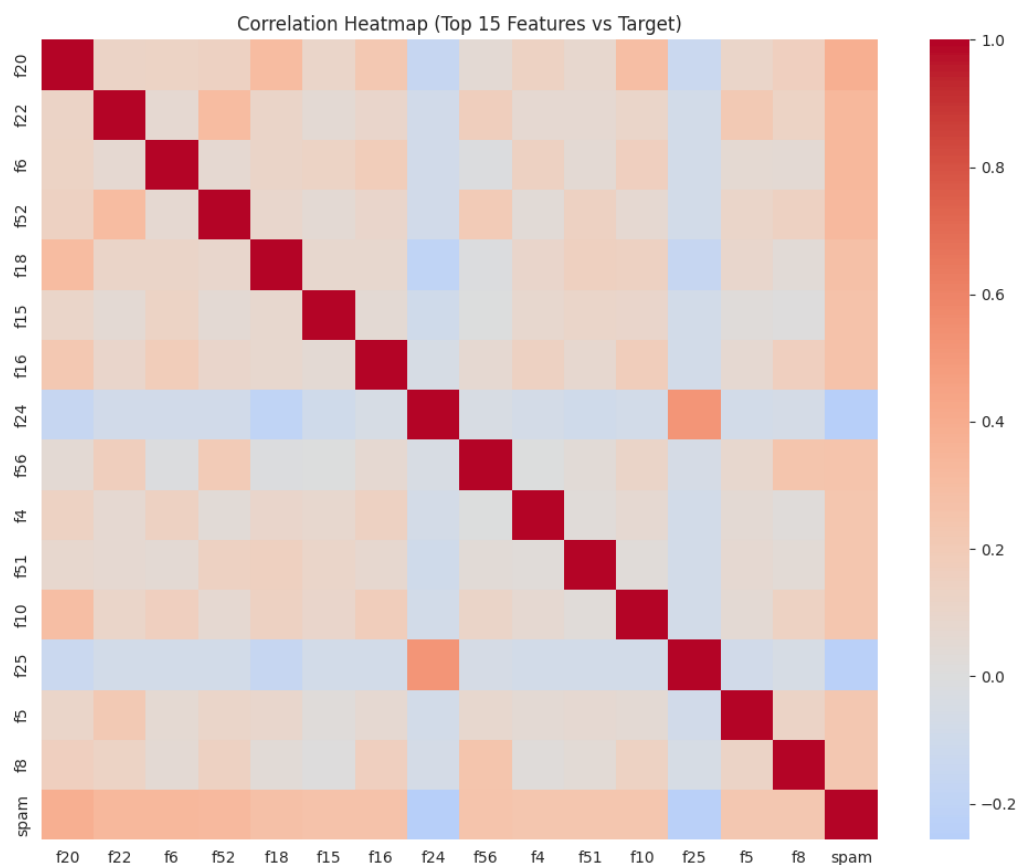


Figure 2: Correlation heatmap of the top 15 features most correlated with the spam target.

Inference: The top 15 features selected by absolute correlation with the target show generally weak-to-moderate positive correlation with `spam` (mostly in the 0.1–0.3 range), with feature `f24` standing out as negatively correlated with the target and with several other top features. No pair of features shows a very strong (near 1.0) correlation, suggesting limited redundancy among the top predictive features and that most of them contribute complementary information.

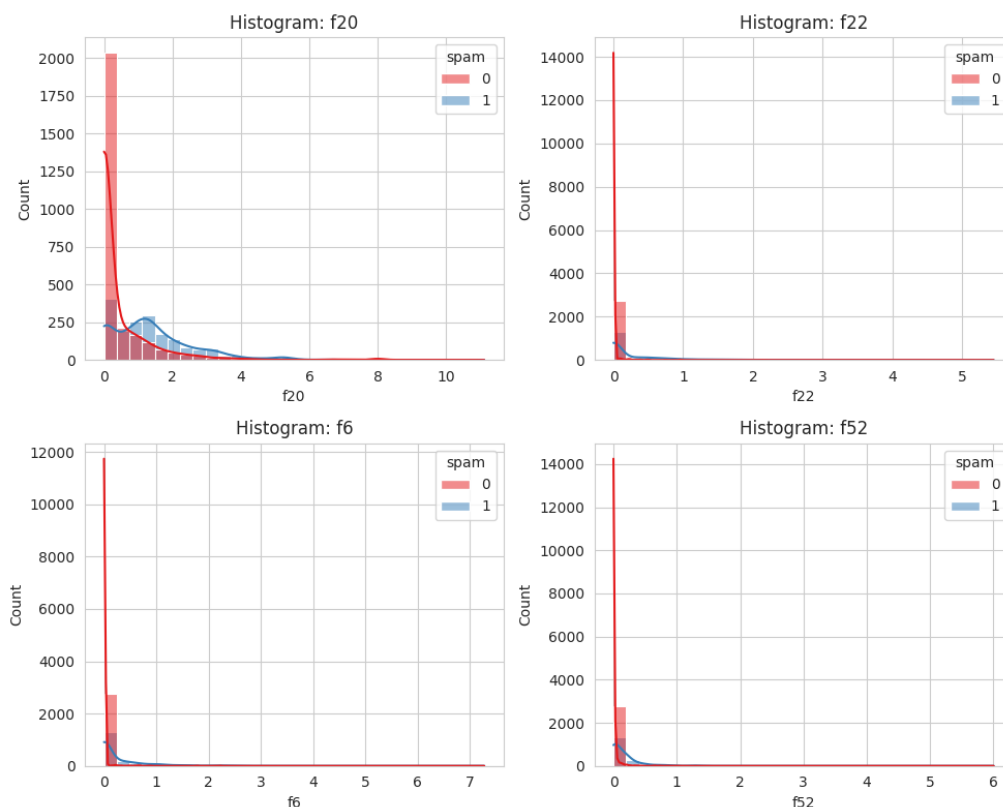


Figure 3: Histograms of the four most target-correlated features, split by class.

Inference: All four features (f20, f22, f6, f52) are heavily right-skewed with most emails having a value near zero. Spam emails (blue) show a visibly wider spread and higher density at larger feature values compared to ham emails (red), confirming these features carry useful discriminative signal. This skew also motivates the use of Multinomial/Bernoulli NB (suited to frequency/count-style data) and feature scaling for Gaussian NB and KNN.

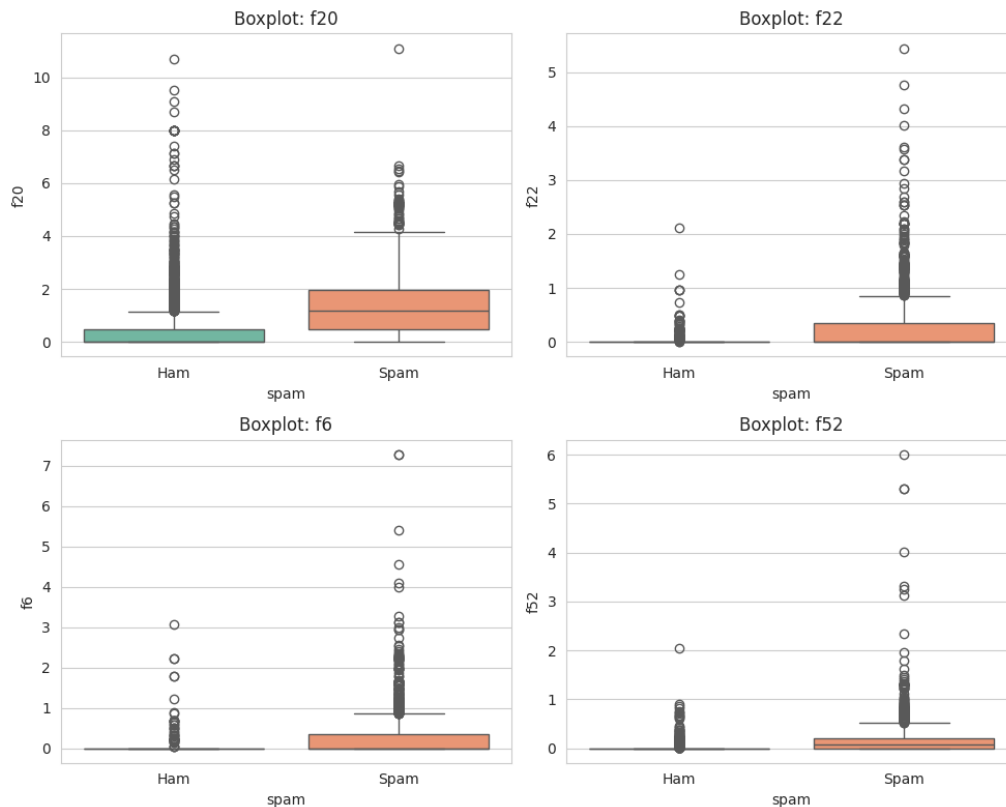


Figure 4: Boxplots of the four most target-correlated features, grouped by class.

Inference: The boxplots confirm that spam emails have a visibly higher median and interquartile range for all four features compared to ham emails, with numerous high-value outliers present in both classes. This separation between class medians supports the predictive value of these features and explains why relatively simple classifiers such as Naïve Bayes and KNN can perform well on this dataset.

5 Data Preprocessing

- **Missing value handling:** No missing values were present in the dataset; the pipeline includes a defensive mean-imputation step (`fillna` with column mean) that was not triggered here since none were found.
- **Encoding:** No categorical encoding was required, as all 57 features are already numeric (frequency/statistics-based).
- **Feature scaling:** Two different scalers were applied depending on model requirements: **StandardScaler** (zero mean, unit variance) was used for Gaussian NB and KNN, which can naturally handle negative/standardized values; **MinMaxScaler** (range $[0, 1]$) was used for Multinomial NB and Bernoulli NB, which require non-negative input features.
- **Train-test split:** An initial stratified 75/25 train-test split (`test_size=0.25`, `random_state=42`) was used for the main comparisons, preserving the original class ratio in both partitions.

- **Feature engineering:** No new derived features were created; all 57 original Spam-base attributes were used directly.

6 Machine Learning Algorithms

Naïve Bayes

- **Working principle:** Applies Bayes' theorem with the "naive" assumption that all features are conditionally independent given the class, and predicts the class with the highest posterior probability.
- **Mathematical formulation:** For a feature vector $X = (x_1, x_2, \dots, x_n)$,

$$\hat{C} = \arg \max_C P(C) \prod_{i=1}^n P(x_i | C) \quad (2)$$

Gaussian NB models $P(x_i | C)$ as a Gaussian distribution; Multinomial NB models it as a multinomial (count-based) distribution; Bernoulli NB models it as a Bernoulli (binary presence/absence) distribution.

- **Advantages:** Extremely fast to train ($O(nd)$) and predict ($O(d)$), performs well even with relatively little training data, and works well for high-dimensional, frequency-style features (Multinomial/Bernoulli variants).
- **Limitations:** The conditional-independence assumption is rarely true in practice, which can hurt calibration of predicted probabilities; Gaussian NB in particular can underperform when features are not normally distributed.

K-Nearest Neighbors (KNN)

- **Working principle:** A lazy learner that stores the training data and, at prediction time, finds the k closest training points to a query (by a distance metric) and assigns the majority class among them.
- **Mathematical formulation:** For a query point x_q , the predicted class is

$$\hat{C} = \text{mode}(\{y_i : x_i \in N_k(x_q)\}) \quad (3)$$

where $N_k(x_q)$ is the set of the k nearest training points to x_q , typically under Euclidean distance $d(x_i, x_q) = \sqrt{\sum_j (x_{ij} - x_{qj})^2}$ or Manhattan distance $d(x_i, x_q) = \sum_j |x_{ij} - x_{qj}|$.

- **Advantages:** Simple, non-parametric (makes no assumption about the underlying data distribution), and can model complex, non-linear decision boundaries.
- **Limitations:** Prediction is computationally expensive for large datasets ($O(nd)$ per query in brute-force form), sensitive to feature scaling and irrelevant features, and requires storing the entire training set in memory.

7 Experimental Setup

Python Version	3.x (Jupyter Notebook environment)
Scikit-Learn Version	As installed in the notebook environment (sklearn)
IDE	Jupyter Notebook
Operating System	Notebook host OS (not explicitly logged in the notebook)
Hardware	Standard CPU (no GPU-specific code used)

8 Hyperparameter Selection

Parameter	Value
KNN k values tested	1, 3, 5, 7, 9, 11
GridSearchCV <code>n_neighbors</code> grid	1, 3, 5, 7, 9, 11, 13, 15
GridSearchCV <code>weights</code> grid	uniform, distance
GridSearchCV <code>algorithm</code> grid	auto, ball_tree, kd_tree
GridSearchCV <code>metric</code> grid	euclidean, manhattan
Cross-validation folds (tuning & evaluation)	5 (Stratified K-Fold)
Train-test split	75% / 25%, stratified, <code>random_state=42</code>

9 Results

Naïve Bayes Comparison

Metric	Gaussian NB	Multinomial NB	Bernoulli NB
Accuracy	0.8254	0.9001	0.8836
Precision	0.7077	0.9472	0.8738
Recall	0.9493	0.7907	0.8238
F1-score	0.8109	0.8619	0.8481
ROC-AUC	0.9368	0.9614	0.9478

KNN Comparison (varying k , StandardScaler features)

k	Accuracy	Precision	Recall	F1
1	0.8975	0.8733	0.8656	0.8695
3	0.9018	0.8814	0.8678	0.8746
5	0.8992	0.8789	0.8634	0.8711
7	0.9053	0.8929	0.8634	0.8779
9	0.9062	0.8932	0.8656	0.8792
11	0.9027	0.8904	0.8590	0.8744

Best k : 9 (highest test accuracy of 0.9062 and highest F1 of 0.8792 among the tested values).

GridSearchCV vs. RandomizedSearchCV

Parameter	GridSearchCV	RandomizedSearchCV
Best k	9	15
Metric	manhattan	manhattan
Weights	distance	distance
Algorithm	auto	auto
CV Accuracy	0.9226	0.9200
Execution Time (s)	50.07	7.40

KDTree vs. BallTree

Metric	KDTree	BallTree
Accuracy	0.9062	0.9062
Training Time (s)	0.0117	0.0086
Prediction Time (s)	0.2412	0.2292

5-Fold Cross Validation

Fold	Naïve Bayes (Multinomial NB)	Best KNN
1	0.8855	0.9130
2	0.8971	0.9290
3	0.8942	0.9377
4	0.8826	0.9232
5	0.8812	0.9130
Average	0.8881	0.9232

Theoretical Time Complexity

Algorithm	Training	Prediction
Naïve Bayes	$O(nd)$	$O(d)$
KNN (Brute)	$O(1)$	$O(nd)$
KDTree	$O(n \log n)$	$O(\log n)_{avg}$
BallTree	$O(n \log n)$	$O(\log n)_{avg}$

Experimental Time Analysis

Algorithm	Training(s)	Prediction(s)
Gaussian NB	0.003457	0.001520
Multinomial NB	0.001931	0.000280
Bernoulli NB	0.003831	0.000969
Best KNN ($k = 9$)	0.011707	0.241179

10 Result Visualization

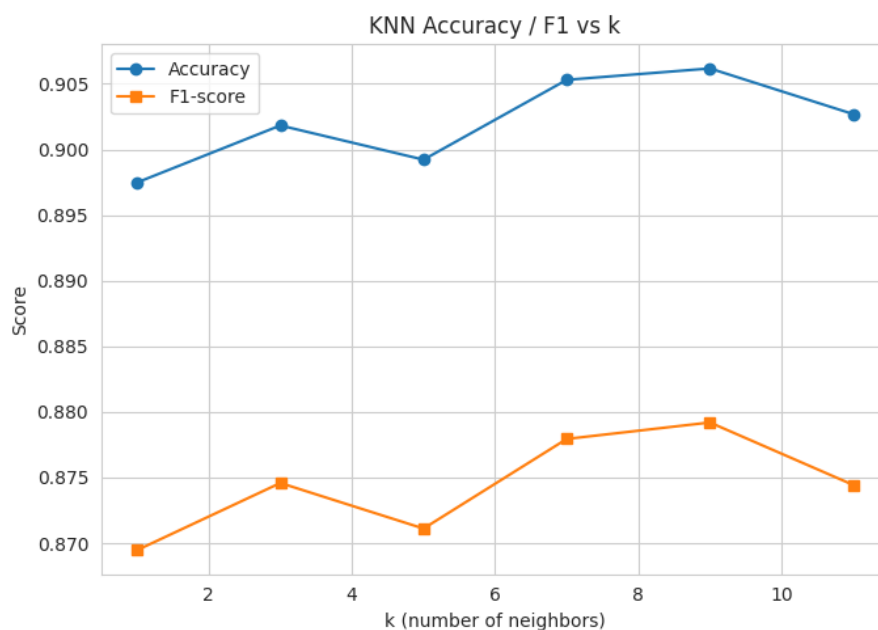


Figure 5: KNN test Accuracy and F1-score as a function of k .

Inference: Accuracy and F1-score both rise from $k = 1$ to a local peak around $k = 3$, dip slightly at $k = 5$, then climb to their maximum near $k = 9$ before declining at $k = 11$. This non-monotonic pattern illustrates the classic bias-variance trade-off in KNN: very small k (e.g., $k = 1$) overfits to noise in individual neighbors, while larger k smooths the decision boundary but can eventually underfit by including less-relevant neighbors. $k = 9$ offers the best balance for this dataset.

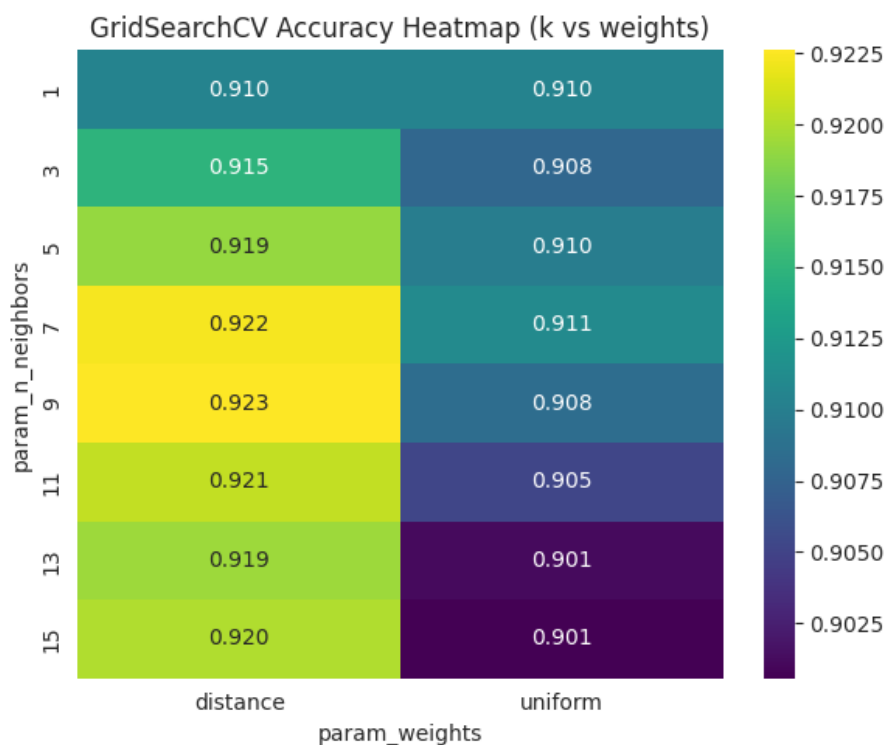


Figure 6: GridSearchCV mean cross-validation accuracy heatmap (k vs. weights, for the best metric = manhattan).

Inference: The heatmap shows that the **distance**-weighted KNN consistently outperforms **uniform** weighting across nearly all values of k , with the best combination at $k = 9$, **weights=distance** (CV accuracy ≈ 0.923). Accuracy generally increases with k up to around 9, then slowly declines, again reflecting the bias-variance trade-off observed in Figure 5.

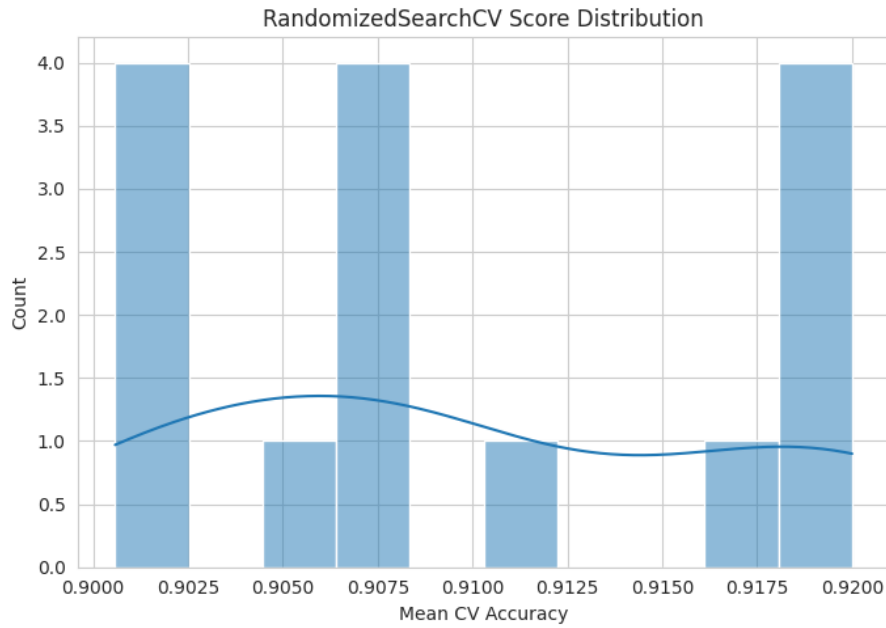


Figure 7: Distribution of mean cross-validation accuracy scores sampled by RandomizedSearchCV.

Inference: The sampled configurations from RandomizedSearchCV cluster tightly between approximately 0.900 and 0.920 mean CV accuracy, showing that KNN performance is fairly stable across a wide range of hyperparameter combinations on this dataset. This also explains why RandomizedSearchCV, despite sampling only a subset of the full grid, was able to find a near-optimal configuration (0.9200 vs. GridSearchCV's 0.9226) in a fraction of the time.

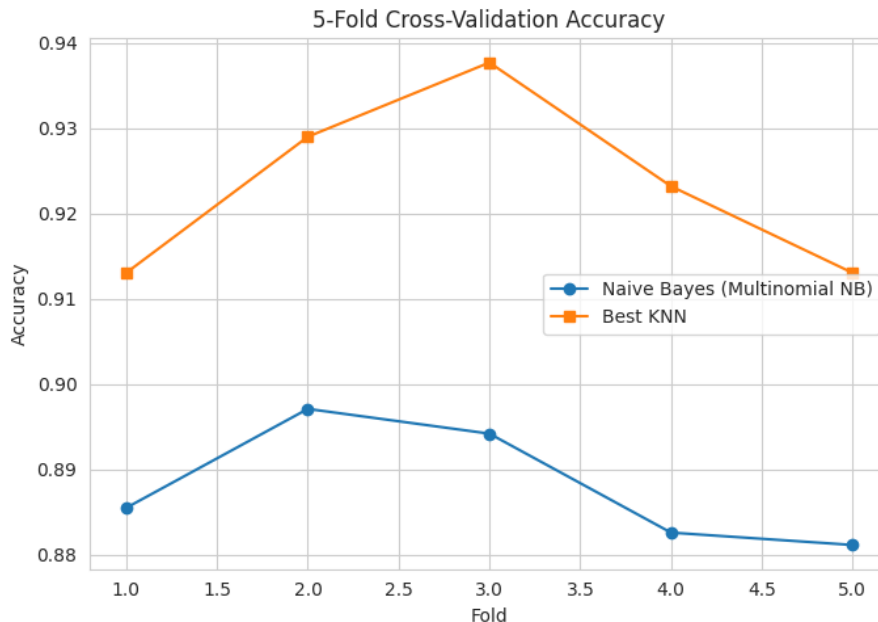


Figure 8: 5-fold cross-validation accuracy per fold for the best Naïve Bayes model (Multinomial NB) and the best KNN model.

Inference: The best KNN model consistently outperforms Multinomial NB across all

5 folds, with both models showing some fold-to-fold variability (KNN ranging from 0.913 to 0.938; Multinomial NB ranging from 0.881 to 0.897). The consistent gap between the two curves indicates that KNN's advantage over Naive Bayes on this dataset is stable and not an artifact of a single lucky train-test split.

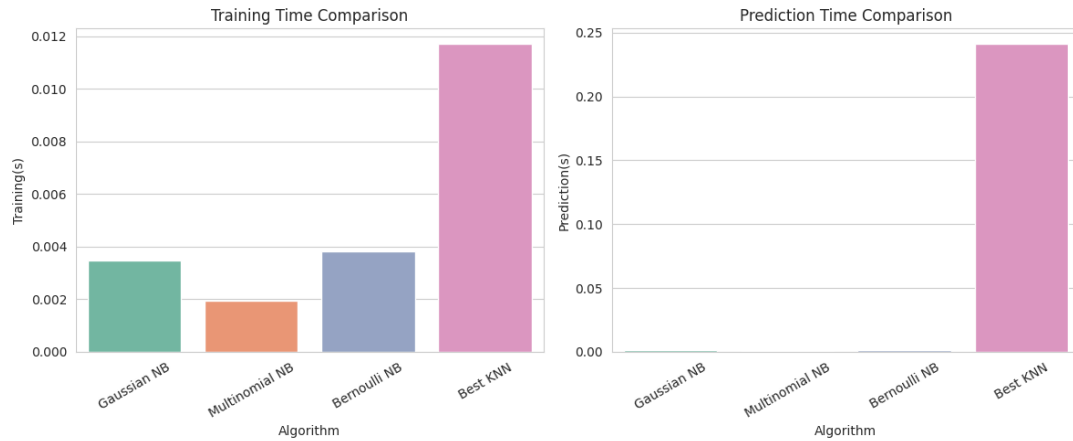


Figure 9: Training time and prediction time comparison across all four models.

Inference: All three Naïve Bayes variants train and predict in a few milliseconds, while KNN's training time is also small (since it only stores/indexes the data) but its prediction time is dramatically higher (over 0.24s on the test set), roughly 100× slower than any Naïve Bayes variant. This empirically confirms the theoretical expectation that KNN shifts computational cost from training to prediction.

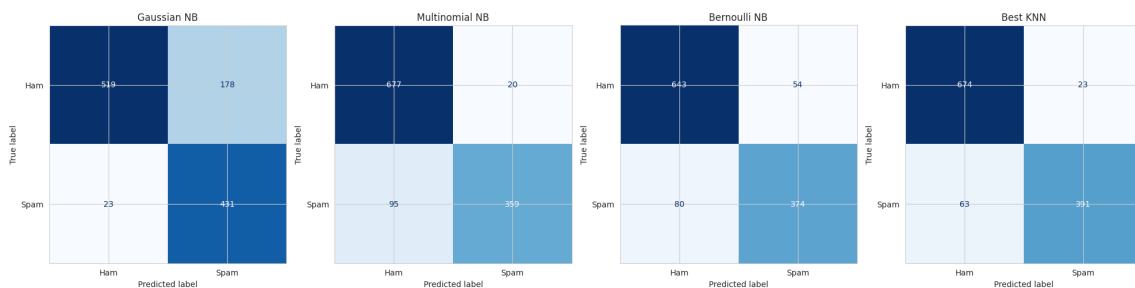


Figure 10: Confusion matrices for Gaussian NB, Multinomial NB, Bernoulli NB, and Best KNN.

Inference: Gaussian NB shows the most false positives (178 ham emails misclassified as spam), consistent with its lower precision. Multinomial NB has very few false positives (20) but more false negatives (95), reflecting its high precision but lower recall. Best KNN achieves the best overall balance, with only 23 false positives and 63 false negatives, consistent with it having the highest accuracy and F1-score among all four models.

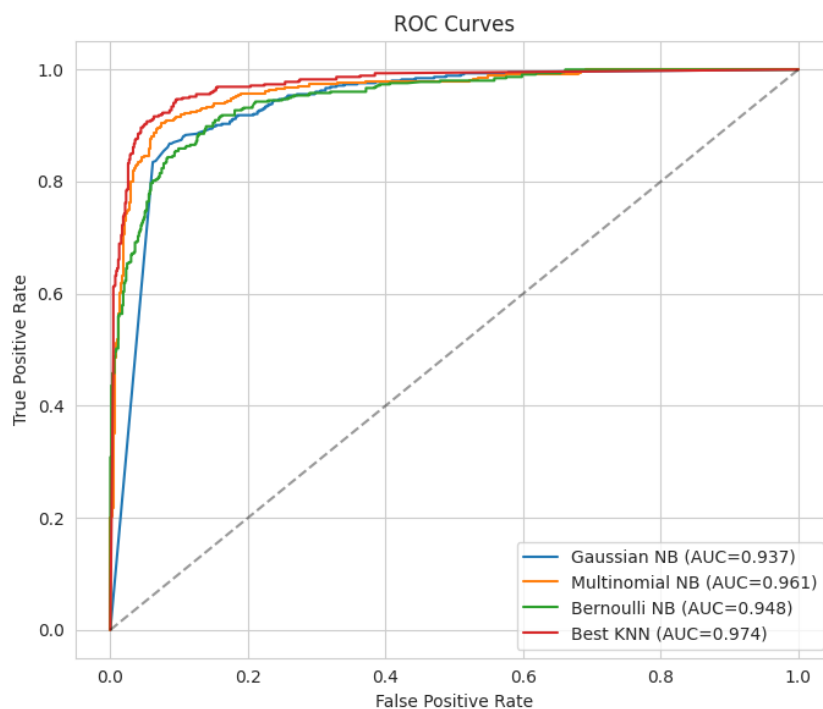


Figure 11: ROC curves for all four classifiers.

Inference: Best KNN achieves the highest AUC (0.974), followed by Multinomial NB (0.961), Bernoulli NB (0.948), and Gaussian NB (0.937). All four curves rise steeply toward the top-left corner, indicating strong overall discriminative ability, but KNN's curve dominates the others across almost the entire false-positive-rate range, confirming it is the strongest classifier by ranking quality as well as by raw accuracy.

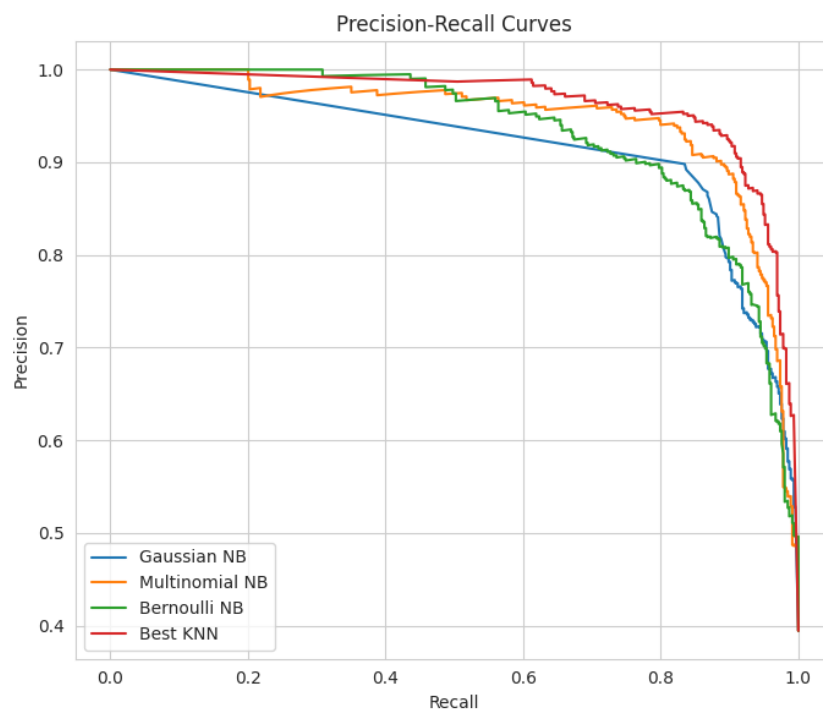


Figure 12: Precision-Recall curves for all four classifiers.

Inference: Bernoulli NB and Multinomial NB maintain very high precision at low recall, but their precision drops off faster than Best KNN's as recall increases. Gaussian NB has the weakest precision-recall trade-off throughout, consistent with its lower F1-score. Best KNN maintains the best precision across most of the recall range, reinforcing its position as the top-performing model on this imbalanced dataset.

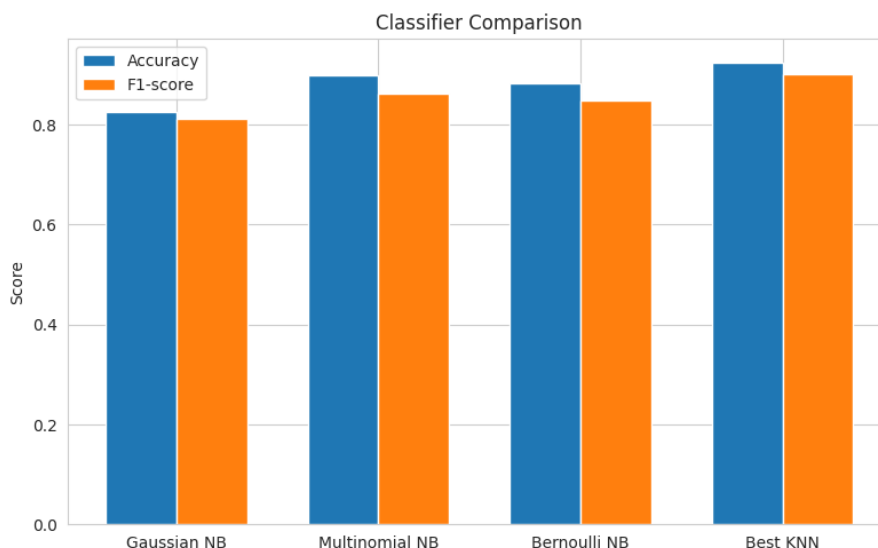


Figure 13: Overall Accuracy and F1-score comparison across all four classifiers.

Inference: Best KNN achieves the highest Accuracy (≈ 0.926) and F1-score (≈ 0.90) among all four models, followed by Multinomial NB. Gaussian NB is the weakest performer on both metrics. This summary chart consolidates the individual metric tables (Section 7) into a single visual comparison, making the overall ranking of classifiers immediately clear: **Best KNN** > **Multinomial NB** > **Bernoulli NB** > **Gaussian NB**.

11 Additional Tasks

Different Train-Test Splits

Test Size	Accuracy
0.20	0.9088
0.25	0.9062
0.30	0.9073
0.40	0.9028

KNN accuracy is fairly stable (within about 0.6 percentage points) across test sizes ranging from 20% to 40%, with the smallest test size (20%, more training data) giving the best accuracy and the largest test size (40%, less training data) giving the lowest, as expected.

Euclidean vs. Manhattan Distance

Metric	Accuracy
Euclidean	0.9062
Manhattan	0.9131

Manhattan distance outperforms Euclidean distance by about 0.7 percentage points at $k = 9$ on this dataset, which is also reflected in GridSearchCV independently selecting `manhattan` as the best metric.

Weighted KNN (Uniform vs. Distance)

Weights	Accuracy
Uniform	0.9062
Distance	0.9140

Distance-weighted voting improves accuracy by about 0.8 percentage points over uniform voting at $k = 9$, since it gives closer neighbors more influence over the predicted class, reducing the impact of farther, potentially less-relevant neighbors.

Overfitting vs. Underfitting in KNN

Small values of k (e.g., $k = 1$) make the model highly sensitive to individual noisy training points, leading to a jagged, overly complex decision boundary that **overfits** the training data – this is reflected in Figure 5, where $k = 1$ has the lowest accuracy and F1 among the small- k values tested. Conversely, very large values of k smooth the decision boundary excessively, causing the model to **underfit** by ignoring local structure in the data and relying on an increasingly global, less discriminative vote; this begins to show as the mild performance drop from $k = 9$ to $k = 11$ in the same figure.

12 Discussion

Among the three Naïve Bayes variants, **Multinomial NB** achieved the highest accuracy (0.900), precision (0.947), F1-score (0.862), and ROC-AUC (0.961), which aligns with theoretical expectations since the Spambase features are frequency/count statistics rather than normally distributed continuous values – the assumption underlying Gaussian NB. Gaussian NB had the highest recall (0.949) but at a steep cost in precision (0.708), meaning it flags many ham emails as spam.

For KNN, accuracy improved as k increased from 1 up to 9, then declined slightly at $k = 11$, and hyperparameter tuning via GridSearchCV confirmed $k = 9$ with Manhattan distance and distance-based weighting as the best configuration (CV accuracy 0.923). RandomizedSearchCV found a comparable configuration (CV accuracy 0.920, with $k = 15$) in roughly 1/7 of GridSearchCV’s execution time (7.4s vs. 50.1s), demonstrating a practical, favorable trade-off between search thoroughness and computational cost when the hyperparameter search space is small enough that a randomized subset still captures near-optimal regions. KDTree and BallTree produced identical accuracy (as expected, since both compute exact nearest neighbors), differing only in construction and query time, with BallTree being marginally faster on this particular run.

Comparing theoretical and empirical complexity: Naïve Bayes' training and prediction times were consistently in the low milliseconds, matching its $O(nd)/O(d)$ theoretical bounds. KNN's training time was also small (index construction only), but its prediction time was roughly $100\times$ larger than any Naïve Bayes variant, empirically confirming the $O(nd)$ per-query prediction cost of brute-force / tree-based KNN search on this dataset size. This makes **Naïve Bayes strongly preferable for large-scale or latency-sensitive deployments**, while KNN remains attractive when maximum predictive accuracy is the priority and the dataset/query volume is manageable.

A limitation of this study is that the dataset was loaded using generic column names (f0–f56) rather than the semantic Spambase attribute names (e.g., `word_freq_free`, `char_freq_!`), since the Kaggle CSV with headers was not available in the working directory and the pipeline fell back to the header-less UCI `.data` file. This does not affect any numerical result but limits direct semantic interpretation of which specific words/characters are most predictive of spam.

13 Conclusion

This experiment successfully implemented and compared Naïve Bayes (Gaussian, Multinomial, Bernoulli) and KNN classifiers on the UCI Spambase dataset (4601 emails, 57 features). **Multinomial NB** was the best-performing Naïve Bayes variant, and **KNN with $k = 9$, Manhattan distance, and distance-weighting** (selected via GridSearchCV) was the overall best-performing model, achieving the highest accuracy (0.926 on 5-fold CV average) and ROC-AUC (0.974) among all four classifiers, at the cost of substantially higher prediction time than any Naïve Bayes variant. GridSearchCV and RandomizedSearchCV converged on similar optimal regions of the hyperparameter space, with RandomizedSearchCV offering a much faster alternative for larger search spaces. KDTree and BallTree produced identical accuracy with comparable speed. Overall, the experiment reinforced key theoretical concepts – the bias-variance trade-off in k , the importance of matching a Naïve Bayes variant's distributional assumption to the data, and the practical training/prediction time trade-off between eager (Naïve Bayes) and lazy (KNN) learners.

14 References

1. F. Pedregosa *et al.*, "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
2. A. Géron, *Hands-On Machine Learning with Scikit-Learn, Keras and TensorFlow*, 2nd ed. O'Reilly Media, 2019.
3. C. M. Bishop, *Pattern Recognition and Machine Learning*. Springer, 2006.
4. M. Hopkins, E. Reeber, G. Forman, and J. Suermondt, "Spambase," UCI Machine Learning Repository, 1999. [Online]. Available: <https://doi.org/10.24432/C53G6X>
5. Kaggle Datasets, "Spambase Dataset," <https://www.kaggle.com/datasets/somesh24/spambase>, accessed 2026.